

**CREDIT RISK PREDICTION VIA STACKED ENSEMBLES
AND BAYESIAN OPTIMIZATION:
A STATISTICALLY VALIDATED CROSS-DATASET
COMPARISON**

*A THESIS SUBMITTED TO CENTRAL UNIVERSITY OF RAJASTHAN
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF*

INTEGRATED MASTER OF SCIENCE

IN

STATISTICS

SUBMITTED BY

TARUN YADAV

Enrollment No.: 2021IMSST021

UNDER THE SUPERVISION OF

Mr. Rahul Singh Chauhan

(Supervisor)

General Manager

Dusker AI

Dr. K. Satish Kumar

(Internal Supervisor)

Assistant Professor

Department of Statistics

Central University of Rajasthan, Rajasthan.



DEPARTMENT OF STATISTICS

CENTRAL UNIVERSITY OF RAJASTHAN, NH-8, BANDARSINDRI, KISHANGARH
(AJMER), RAJASTHAN -305817, INDIA

2026



TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Tarun Yadav**, Enroll. No. 2021IMSST021, Int. M.Sc. Statistics student of the Department of Statistics has worked on a project titled “*Credit Risk Prediction via Stacked Ensembles and Bayesian Optimization: A Statistically Validated Cross-Dataset Comparison*” under the guidance of **Mr. R. S. Chauhan**, General Manager, Dusker AI, towards the partial fulfillment of the award of “Integrated Master of Science in Statistics” during the period July 2021 to May 2026.

Place: Central University of Rajasthan, Ajmer

Date:

Dr. Deepesh Bhati

Head

Department of Statistics

Central University of Rajasthan

Dr. K. Satish Kumar

Assistant Professor

Department of Statistics

Email : satish.kumar@curaj.ac.in

Website: <http://www.curaj.ac.in>



Central University of Rajasthan

(A Central University by an Act of Parliament)

NH-8, Bandarsindri, 305817

Kishangarh (Ajmer), Rajasthan, INDIA

Phone (Office): +91-1463-238755 / 260200

Telefax: +91-1463-238722

Certificate

This is to certify that the work embodied in the accompanying project report entitled *“Credit Risk Prediction via Stacked Ensembles and Bayesian Optimization: A Statistically Validated Cross-Dataset Comparison”*, submitted by *Mr. Tarun Yadav*, Xth Semester Int. M.Sc. Statistics student of the Department of Statistics, Central University of Rajasthan, under my supervision.

Place: Central University of Rajasthan, Ajmer

Date:

Dr. K. Satish Kumar

(Internal Supervisor)

Assistant Professor

Department of Statistics



DUSKER AI

☎ +91 70618477192

✉ rahul.singh@dusker.ai

🌐 www.dusker.ai

21st May 2026

INTERNSHIP CERTIFICATE

This is clarify that **Mr. Tarun Yadav (2021IMSST021)**, a student of **Central University Of Rajasthan**, pursued an internship with **Dusker AI** from **20th January 2026** to **20th May 2026**. During this period, he worked sincerely and actively contributed to the organization as an intern.

During his internship as a **Data Analyst and AI Research Intern**, he worked on **Credit Risk Prediction via Stacked Ensembles and Bayesian Optimization: A Statistically Validated Cross-Dataset Comparison** in which he built and evaluated a machine-learning system that helps predict whether a loan applicant is likely to default. He compared eight different prediction models ranging from simple statistical methods to advanced ensemble techniques on four publicly available credit datasets drawn from banks across the world. Rather than relying on any one model, his approach combined several of them into a single stacked ensemble, similar to polling a panel of experts and weighing their opinions to arrive at a more reliable conclusion. This combined system was the most consistent performer across every dataset tested, outperforming every individual model on the headline measures. He also verifieth the improvement using formal statistical tests rather than relying on raw numbers alone. The resulting pipeline is reproducible, well documented, and directly applicable to credit-scoring problems faced by lenders today.

Tarun demonstrated dedication, professionalism, and a strong willingness to learn throughout the internship. His performance has been commendable and he made valuable contributions to ongoing projects. He consistently showed commitment and sincerity towards her responsibilities and successfully completed her internship with the organization.

We wish him continued success in his future endeavors.

Regards

Rahul Singh Chauhan

General Manager

Dusker AI





Declaration

I, **Tarun Yadav**, hereby declare that the project work entitled “*Credit Risk Prediction via Stacked Ensembles and Bayesian Optimization: A Statistically Validated Cross-Dataset Comparison*” submitted to the Department of Statistics, Central University of Rajasthan as a partial fulfilment of requirements of X-Semester examination, is a bona fide record of work undertaken by me, under the supervision of **Mr. R. S. Chauhan**, General Manager, Dusker AI, and it has not formed the basis for the award of any other Degree/Associateship/Fellowship by any University.

Signature of Candidate

Tarun Yadav

Enroll. No.: 2021IMSST021

Place: Bandarsindri

Date:

Acknowledgement

I would like to express my sincere gratitude to all those who have supported and guided me throughout the journey of completing this thesis.

First, I am deeply grateful to my supervisor, **Mr. R. S. Chauhan**, General Manager, Dusker AI, for his invaluable guidance, unwavering support, and insightful feedback. His expertise and dedication played a pivotal role in shaping the direction of my research and enhancing the quality of this work.

My heartfelt thanks go to my family and my friend Mr. D. D. N. S. Chandrasekhar Sharma for their constant encouragement and steady support throughout this work.

Thank you all for being a part of this fulfilling endeavor.

Contents

1	Introduction	1
1.1	Background and Motivation	2
1.2	Problem Statement and Objectives	3
1.3	Contributions	4
1.4	Organisation of the Thesis	4
2	Classification Algorithms	6
2.1	Logistic Regression	6
2.2	k -Nearest Neighbours	7
2.3	Decision Tree	7
2.4	Support Vector Machine	8
2.5	Random Forest	8
2.6	Gradient Boosting	8
2.7	XGBoost	9
2.8	Stacked Ensemble	9
2.8.1	Out-of-fold Meta-features	9
2.8.2	Meta-learner Training	10
2.8.3	Why Stacking Works	10
3	Methodology	12
3.1	Feature Selection	12
3.1.1	Selected Feature Counts per (Dataset, FS) Pair	13
3.1.2	Selected Features per Dataset	13
3.2	Class Imbalance	17
3.2.1	SMOTE Oversampling	17

3.2.2	GHOST Threshold Tuning	17
3.3	Hyperparameter Tuning by Bayesian Optimisation	18
3.3.1	Surrogate Model	18
3.3.2	Acquisition Function	18
3.3.3	Objective and Budget	19
3.4	Stacked Ensemble Configuration	19
3.4.1	Base Layer (Five Classifiers)	19
3.4.2	Meta-Feature Matrix	20
3.4.3	Meta Layer (Five Candidates)	20
3.5	GA-MaxAccLogit Meta-learner	21
3.5.1	The ProfLogit Reference Algorithm	21
3.5.2	The Adaptation: GA-MaxAccLogit	21
3.5.3	Algorithm	21
3.5.4	Justification of the Replacement	22
4	Experimental Setup	23
4.1	Datasets	23
4.2	Preprocessing	24
4.3	Evaluation Metrics	25
4.3.1	Headline Metrics	25
4.3.2	Ten-measure Suite	25
4.4	Implementation	27
4.5	Cross-validation Protocol	28
5	Results and Statistical Analysis	29
5.1	Per-dataset Leaderboards	29
5.1.1	Eleven-Metric Per-Classifer Scoreboard	30
5.2	Cross-dataset Comparison	32
5.3	Friedman Test	34
5.3.1	Blocks and Ranks	34
5.3.2	Friedman Statistic	35
5.4	Pairwise Post-hoc: Wilcoxon–Holm	36
5.4.1	Procedure	36

5.4.2	Results on F_1 -macro	36
5.4.3	Results on AUC	37
5.4.4	Results on G-Mean	38
5.5	Discussion	38
6	Conclusion	39
6.1	Summary of Findings	39
6.2	Contributions Revisited	40
6.3	Why the Stacked Ensemble is the Recommended Model	41
6.4	Limitations	42
6.5	Future Work	43
A	Python Code Listings	45
A.1	Friedman Test on 16 Blocks	45
A.2	Wilcoxon Signed-Rank with Holm Correction	46
A.3	GA-MaxAccLogit Meta-learner	47
A.4	Stacked Ensemble with Best-Meta Selection	50

Chapter 1

Introduction

Credit risk prediction is one of the oldest and most economically consequential applications of statistical learning. Whenever a lender chooses whether to extend credit to an applicant, the decision is backed — explicitly or implicitly — by a model that assigns each applicant a probability of default. A classifier that wrongly clears a defaulter incurs a direct monetary loss, while one that wrongly rejects a creditworthy applicant loses future revenue and damages the lender’s competitiveness. The two errors are not symmetric in cost, yet both must be controlled at the same time. Because the consequences of a misclassification fall on real money, the development of credit-risk classifiers has long served as a proving ground for new statistical and machine-learning ideas.

The literature on credit scoring covers a wide spectrum of methods: classical linear discriminant analysis and logistic regression, non-parametric methods such as k -nearest neighbours, decision trees and rule-based systems, and modern ensembles and neural networks [1, 2]. Across this body of work two observations recur:

1. No single algorithm is uniformly best across all credit datasets; the winner depends on dataset size, class balance, feature mix and the metric of interest.
2. Combining several heterogeneous classifiers — through bagging, boosting or stacked generalisation — tends to produce more robust performance than any single learner alone [3].

Both observations motivate the present work. We do not propose a single new classifier; rather, we build a *stacked ensemble* of existing classifiers, select its second-level (meta) learner by an objective rule, and submit the entire benchmark to non-parametric significance testing

across multiple datasets so that the empirical ordering of the classifiers is supported by formal statistics rather than by raw accuracy figures alone.

1.1 Background and Motivation

A credit-risk classifier maps an applicant’s covariate vector $\mathbf{x} \in \mathbb{R}^p$ to a binary label $y \in \{0, 1\}$ (default or no default). In practice the classifier produces a score $s(\mathbf{x}) \in [0, 1]$ interpreted as the estimated probability of default; a threshold t is then applied to obtain a crisp class label. Although this formulation is simple, three issues make credit datasets challenging in practice:

Class imbalance. Defaulters are typically the minority class. A naive classifier that predicts the majority label for every applicant achieves high accuracy but is useless for the lender, who specifically needs to identify the minority of likely defaulters. The standard responses are either to resample the training data — for example by the *SMOTE* oversampling scheme of Chawla et al. [4] — or to tune the decision threshold t explicitly, as in the *GHOST* procedure of Esposito et al. [5].

Mixed feature types. Credit applications contain numeric attributes (age, income, debt ratio) alongside categorical ones (occupation, marital status, residence type). A robust pipeline needs both an encoding scheme that respects this distinction and a scaling step that prevents the numeric attributes from dominating distance-based classifiers such as the k -nearest-neighbours rule.

Threshold and metric sensitivity. Several widely-reported metrics — accuracy, F_1 , Cohen’s κ — depend on a chosen threshold t . Two classifiers can produce identical area under the ROC curve (AUC), which is threshold-independent, and yet report very different accuracies because their default thresholds happen to favour different operating points. This is a known source of confusion in the benchmark literature, and our evaluation explicitly reports both threshold-dependent and threshold-independent metrics to guard against it.

A further complication, often glossed over in single-dataset studies, is statistical: when several classifiers are compared on several datasets, the differences in observed performance must be assessed relative to the variability across datasets, not in absolute terms. Demšar [6] laid out the now-standard non-parametric procedure for this purpose: a Friedman rank test followed by an appropriate post-hoc correction — typically Holm [7] or Nemenyi — to pin down which pairs of classifiers are significantly different. The present thesis applies this

protocol to a four-dataset benchmark.

1.2 Problem Statement and Objectives

The empirical question of this thesis is straightforward:

Among a representative set of credit-risk classifiers, which model performs most consistently across multiple available UCI datasets, and is its advantage statistically significant relative to the others?

The answer is operationalised by the following objectives:

1. Implement eight representative classifiers — Logistic Regression, k -Nearest Neighbours, Support Vector Machine (RBF kernel), Decision Tree, Random Forest, Gradient Boosting, XGBoost, and a Stacked Ensemble — and apply them uniformly to four UCI credit datasets (Australian, Japanese, German, Taiwan).
2. Build a feature-selection sweep with four strategies (Full features, Recursive Feature Elimination, L1-penalised Logistic Regression, Forward Selection) per dataset, so that the comparison spans both “unselected” and “selected” feature regimes.
3. Make the Stacked Ensemble’s second-level learner a choice from five candidates (Logistic Regression, Random Forest, XGBoost, Multi-Layer Perceptron, and a Genetic-Algorithm-tuned logistic regression, GA-MaxAccLogit), and select the meta by an objective F_1 -macro rule on k -fold out-of-fold predictions. The GA-MaxAccLogit meta is adapted from the ProfLogit method of Stripling et al. [8]: we keep ProfLogit’s genetic machinery but replace its expected-profit objective with F_1 -macro.
4. Tune Random Forest and XGBoost by *Bayesian optimisation* (Gaussian-process surrogate, Expected-Improvement acquisition) so that the heavyweight learners are not unfairly penalised by default settings [9].
5. Apply *SMOTE* to the training data of every imbalanced dataset and tune the decision threshold via *GHOST* on the same datasets, to guarantee that the minority class is not silently ignored by the threshold-dependent metrics.
6. Evaluate every (dataset, feature-selection, classifier) triple on the held-out test set using F_1 -macro, AUC and a ten-measure diagnostic suite, and report the full leaderboard.

7. Test the global null hypothesis “all eight classifiers perform equally” by the *Friedman* rank test [10] across the sixteen evaluation blocks (4 datasets $\times$ 4 feature-selection variants), and resolve the pairwise comparisons by the *Wilcoxon signed-rank* test with *Holm* step-down correction [6].

Objectives 1–6 construct the benchmark; objective 7 elevates the comparison from descriptive to inferential. The combination is what distinguishes the present thesis from many credit-scoring benchmarks in the literature.

1.3 Contributions

The principal contributions of this thesis are the following:

1. A *uniform pipeline* that applies the same preprocessing, feature-selection sweep, hyperparameter tuning, and evaluation suite to all eight classifiers on all four UCI datasets, producing a leaderboard of $4 \times 4 \times 8 = 128$ model rows that is internally consistent.
2. An adaptation of the ProfLogit genetic-algorithm logistic model [8], here called **GA-MaxAccLogit**, in which the expected-profit fitness function is replaced by the classification metric F_1 -macro (penalised by an L_1 norm on the coefficients). The adapted learner is plugged in as a candidate meta-learner of the stacked ensemble.
3. A statistical-significance layer using *Friedman* + *Wilcoxon-Holm* on the rank matrix of the sixteen (dataset $\times$ FS) blocks, treating each block as an independent evaluation. This is the methodology of Demšar [6] applied at the block — not dataset — level, and it yields markedly more power than the equivalent four-dataset test.
4. A reproducible *Colab notebook bundle* containing the Friedman test, the pairwise Wilcoxon-Holm post-hoc, the GA-MaxAccLogit implementation, and the data-loading scripts — all runnable end-to-end on the open UCI data.

1.4 Organisation of the Thesis

The remainder of this thesis is organised as follows.

Chapter 2 reviews the eight classifiers used in the benchmark and develops the mathematical formulation of each. The stacked ensemble is introduced at the end of the chapter as

the formal meta-learner setup that ties the previous seven together.

Chapter 3 describes the methodological contributions: the four feature-selection strategies, the SMOTE + GHOST imbalance pipeline, the Bayesian-optimisation tuning protocol, and the GA-MaxAccLogit meta-learner adapted from ProfLogit.

Chapter 4 describes the four UCI datasets, the preprocessing pipeline, the evaluation metrics, and the implementation environment.

Chapter 5 reports the empirical results: the per-dataset leaderboards, the cross-dataset comparison, the Friedman test, and the Wilcoxon-Holm pairwise post-hoc. The chapter ends with a discussion of why the Stacked Ensemble is the most consistent performer.

Chapter 6 concludes the thesis, discusses the limitations of the present study, and outlines directions for future work. Appendix A reproduces the principal Python code listings used in the experiments.

Chapter 2

Classification Algorithms

This chapter reviews the eight classifiers that make up the benchmark of the thesis. Two of them — Logistic Regression and k -Nearest Neighbours — are classical statistical methods. Five — Decision Tree, Support Vector Machine, Random Forest, Gradient Boosting and XGBoost — are machine-learning models. The eighth, the Stacked Ensemble, combines the others through a meta-learner; it is introduced formally in Section 2.8.

Throughout the chapter let $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ denote the training set, with covariate vector $\mathbf{x}_i \in \mathbb{R}^p$ and binary label $y_i \in \{0, 1\}$, where $y = 1$ encodes default.

2.1 Logistic Regression

Logistic regression is a generalised linear model with a binomial response and a logit link function. It models the conditional probability of default as

$$P(y_i = 1 \mid \mathbf{x}_i) = \sigma(\beta_0 + \boldsymbol{\beta}^T \mathbf{x}_i), \quad \sigma(t) = \frac{1}{1 + e^{-t}}, \quad (2.1)$$

where $\beta_0 \in \mathbb{R}$ is the intercept and $\boldsymbol{\beta} \in \mathbb{R}^p$ is the vector of regression coefficients. Estimation is by maximum likelihood: the parameters $(\beta_0, \boldsymbol{\beta})$ are chosen to maximise the log-likelihood

$$\ell(\beta_0, \boldsymbol{\beta}) = \sum_{i=1}^N \{y_i \log \sigma(\eta_i) + (1 - y_i) \log(1 - \sigma(\eta_i))\}, \quad \eta_i = \beta_0 + \boldsymbol{\beta}^T \mathbf{x}_i. \quad (2.2)$$

Logistic regression imposes a linear decision boundary in the original feature space; its strength is interpretability (each coefficient β_j has the log-odds interpretation), and its weakness is its inability to capture non-linear interactions without explicit feature engineering. In this thesis

it serves as the classical statistical baseline.

2.2 k -Nearest Neighbours

The k -nearest-neighbours rule is the simplest non-parametric classifier: it stores the entire training set D and, at prediction time for a new applicant $\mathbf{x}^*$, retrieves the k training points closest to $\mathbf{x}^*$ under a chosen distance and assigns the majority class among them.

Let $d(\mathbf{x}^*, \mathbf{x}_i)$ denote the Euclidean distance between $\mathbf{x}^*$ and $\mathbf{x}_i$, and let $\mathcal{N}_k(\mathbf{x}^*)$ be the set of indices of the k training points with the smallest $d(\mathbf{x}^*, \cdot)$. Then

$$\hat{y}(\mathbf{x}^*) = \text{mode}\{y_i : i \in \mathcal{N}_k(\mathbf{x}^*)\}. \quad (2.3)$$

k -NN is sensitive to the scale of the features — the Euclidean distance is dominated by attributes with large numerical range — so features are standardised before training. The hyperparameter k trades off variance (k small) against bias (k large); we use $k = 5$ throughout, which is the common default in scikit-learn.

2.3 Decision Tree

A decision tree partitions the feature space by a sequence of axis-aligned binary splits, fitted greedily by maximising an impurity reduction criterion at each node. Let T be a tree, let $\mathcal{R}_m \subset \mathbb{R}^p$ be the hyperrectangle associated with the m -th leaf, and let $p_{m,c}$ denote the empirical class proportion in that leaf:

$$p_{m,c} = \frac{1}{N_m} \sum_{i: \mathbf{x}_i \in \mathcal{R}_m} \mathbb{I}(y_i = c), \quad c \in \{0, 1\}, \quad (2.4)$$

where N_m is the number of training points falling into leaf m . The Gini impurity of leaf m is

$$G(m) = \sum_{c=0}^1 p_{m,c}(1 - p_{m,c}) = 1 - p_{m,0}^2 - p_{m,1}^2. \quad (2.5)$$

Splits are chosen to minimise the weighted sum of child impurities, i.e. to maximise $G(\text{parent}) - (N_L G(L) + N_R G(R))/N$ over all possible split positions on each feature, where L and R are the left and right children of the current node. The tree is grown until a minimum-leaf or maximum-depth stopping criterion is satisfied; in this thesis we keep the scikit-learn defaults

except for setting a fixed random seed.

2.4 Support Vector Machine

The Support Vector Machine finds the linear hyperplane that maximises the margin between the two classes in a feature space possibly transformed by a kernel. For a feature map $\phi : \mathbb{R}^p \rightarrow \mathcal{H}$ inducing a kernel $k(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$, the soft-margin SVM solves

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad \text{s.t.} \quad \tilde{y}_i (\mathbf{w}^T \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad (2.6)$$

where $\tilde{y}_i = 2y_i - 1 \in \{-1, +1\}$, $C > 0$ controls the trade-off between margin width and misclassification, and the slack variables ξ_i allow individual training points to lie inside the margin. In this thesis we use the Gaussian (radial-basis-function) kernel

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\right), \quad (2.7)$$

which allows non-linear decision boundaries while retaining the maximum-margin formulation.

2.5 Random Forest

A Random Forest [1] is an ensemble of B decision trees grown on bootstrap samples of the training data, with the additional randomisation that at each split only a random subset of $m \leq p$ features is considered for the splitting decision. Let $\hat{T}^{(b)}(\mathbf{x})$ denote the prediction of the b -th tree. The forest's prediction is the majority vote

$$\hat{y}_{\text{RF}}(\mathbf{x}) = \text{mode}\{\hat{T}^{(1)}(\mathbf{x}), \dots, \hat{T}^{(B)}(\mathbf{x})\}. \quad (2.8)$$

The bootstrap sampling and random feature subsetting decorrelate the individual trees so that their averaged variance is smaller than the variance of any single tree. In our experiments $B = 300$ and $m = \sqrt{p}$.

2.6 Gradient Boosting

Gradient Boosting also builds an ensemble of trees, but the trees are added sequentially, each fitting the *pseudo-residuals* of the previous ensemble. Starting from a constant predictor

$F_0(\mathbf{x})$, the m -th boosting iteration solves

$$h_m = \arg \min_h \sum_{i=1}^N (r_{i,m} - h(\mathbf{x}_i))^2, \quad r_{i,m} = - \left. \frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right|_{F=F_{m-1}}, \quad (2.9)$$

where L is the log-loss for binary classification and h ranges over depth-limited regression trees. The new predictor is $F_m = F_{m-1} + \nu h_m$, with shrinkage factor $\nu \in (0, 1]$. After M boosting iterations the final classifier converts $F_M(\mathbf{x})$ to a probability via the logistic transformation $\sigma(F_M(\mathbf{x}))$.

2.7 XGBoost

XGBoost [2] is a regularised, second-order implementation of gradient boosting that has become the most popular single-classifier choice for tabular benchmarks. Its objective at iteration m includes both the loss and an explicit complexity penalty:

$$\mathcal{L}^{(m)} = \sum_{i=1}^N L(y_i, F_{m-1}(\mathbf{x}_i) + h_m(\mathbf{x}_i)) + \Omega(h_m), \quad \Omega(h) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2, \quad (2.10)$$

where T is the number of leaves of h_m , $\mathbf{w}$ is the vector of leaf scores, and $\gamma, \lambda \geq 0$ are regularisation hyperparameters. XGBoost uses a second-order Taylor expansion of L to compute the leaf scores and split gains in closed form, which yields both faster training and tighter regularisation than the first-order gradient-boosting recursion.

2.8 Stacked Ensemble

Stacked Generalisation, introduced by Wolpert [3], is a meta-learning method in which an additional model — the *meta-learner* g — is trained to combine the predictions of several *base learners* $h_1, \dots, h_M$. The principal challenge is to avoid *data leakage*: if the meta-features were the in-sample predictions of the base learners, the meta would be trained on values that are already too optimistic.

2.8.1 Out-of-fold Meta-features

Stacking solves the leakage problem by k -fold out-of-fold (OOF) prediction. Partition the training set into k folds. For each fold j and each base learner h_m :

1. Train h_m on the data of all folds except j .
2. Predict on the held-out fold j .

Every training point thus receives one prediction from a base learner that did not see it during training. Stacking these predictions across the M base learners gives the *meta-feature matrix*

$$\mathbf{z}_i = (h_1^{(-)}(\mathbf{x}_i), h_2^{(-)}(\mathbf{x}_i), \dots, h_M^{(-)}(\mathbf{x}_i))^T \in \mathbb{R}^M, \quad (2.11)$$

where the superscript $(-)$ indicates the OOF prediction.

2.8.2 Meta-learner Training

The meta-learner g is then estimated on the pairs $\{(\mathbf{z}_i, y_i)\}_{i=1}^N$ by minimising a loss L_{meta} :

$$g^* = \arg \min_g \sum_{i=1}^N L_{\text{meta}}(y_i, g(\mathbf{z}_i)). \quad (2.12)$$

At prediction time, each base learner is refit on the *whole* training set, the new applicant $\mathbf{x}^*$ is pushed through all the base learners to obtain a fresh meta-feature vector $\mathbf{z}^*$, and the stacked prediction is $\hat{y}_{\text{stack}}(\mathbf{x}^*) = g^*(\mathbf{z}^*)$.

The full pipeline is summarised in Figure 3.1 (Chapter 3), where the specific choice of base and meta learners used in this thesis is also reported. We defer that material to Chapter 3 because the meta-selection rule and the GA-MaxAccLogit candidate are themselves methodological contributions of the work.

2.8.3 Why Stacking Works

The expected error of a classifier can be decomposed into the sum of its squared bias and its variance. Averaging the predictions of diverse classifiers — as bagging does — reduces the variance term but leaves the bias unchanged. Stacking, in contrast, lets the meta-learner *re-weight* the base learners in a region-dependent way: when the OOF predictions reveal that one base learner is more accurate in a certain area of feature space, the meta-learner can lean on that base learner there and on a different base learner elsewhere. In effect the meta-learner can correct systematic bias of any individual base learner so long as the bias is detectable from the OOF predictions of the other base learners. This is the intuition behind the empirical

observation, confirmed in Chapter 5, that the stacked ensemble is the only classifier in the benchmark that is never the weak link on any of the four datasets.

Chapter 3

Methodology

The previous chapter introduced the eight individual classifiers. This chapter describes the methodological framework that ties them into a single, reproducible pipeline. The framework has five ingredients:

- a sweep of four *feature-selection* strategies (Section 3.1),
- a class-imbalance handling layer combining *SMOTE* and *GHOST* threshold tuning (Section 3.2),
- a *Bayesian-optimisation* hyperparameter-tuning protocol for the heavyweight learners (Section 3.3),
- a *stacked-ensemble configuration* with five candidate meta-learners and an objective best-meta selection rule (Section 3.4),
- and the *GA-MaxAccLogit* meta-learner — an adaptation of the ProfLogit method of Stripling et al. [8] — described in Section 3.5.

Each subsection states the choice we make, the rationale, and the specific hyperparameters used in the experiments.

3.1 Feature Selection

A feature-selection step is included for two reasons: noise reduction in the input space tends to improve the generalisation of distance- and kernel-based classifiers, and the empirical pattern of which features each algorithm “wants” is itself of analytic interest. Rather than commit to a single selection strategy, we sweep four:

Full. All features are kept, after one-hot encoding the categorical attributes. This is the no-selection baseline against which the other three are measured.

Recursive Feature Elimination (RFE). Starting from the full feature set, RFE iteratively trains a base classifier, ranks the features by the magnitude of the classifier’s coefficient or feature importance, and discards the lowest-ranked feature. We use logistic regression as the underlying classifier and fix the number of features to retain at $p/2$, rounded.

Lasso-LR. An L_1 -penalised logistic regression is fitted on the training data; only the features with non-zero coefficients are retained. The penalty strength λ_{L1} is chosen by 5-fold cross-validation on the same training data.

Forward Selection. Starting from the empty feature set, features are added one at a time in the order that maximises the F_1 -macro of a logistic-regression classifier on 5-fold cross-validation. Selection stops when adding the next feature no longer improves the cross-validated score by more than a small tolerance.

For each (dataset, feature-selection) pair, all eight classifiers are trained and evaluated separately. The benchmark thus contains $4 \text{ datasets} \times 4 \text{ FS} \times 8 \text{ models} = 128$ classifier rows.

3.1.1 Selected Feature Counts per (Dataset, FS) Pair

Table 3.1 reports the number of features each method keeps on each of the four datasets, starting from the one-hot-encoded input. The Full column is the post-encoding size of the feature matrix (which differs from the raw attribute count when categorical attributes are present).

Table 3.1: Number of features retained by each FS method on each dataset (post one-hot encoding). Full = baseline; RFE/Forward retain $\lceil p/2 \rceil$; Lasso-LR retains all features with non-zero L_1 coefficient.

Dataset	Full	RFE	Lasso-LR	Forward
Australian	14	7	12	7
Japanese	37	18	32	18
German	48	24	46	24
Taiwan	23	11	21	11

3.1.2 Selected Features per Dataset

Tables 3.2–3.5 below list, for each dataset, the exact feature each method kept (✓) or dropped (—). The Full column is omitted because it always retains every feature. Feature names are reported as they appear after one-hot encoding; for the German dataset, dummy variables

Table 3.5: Taiwan Default of Credit Card Clients — features retained (✓) or dropped (—) by each FS method. Variables follow the original UCI naming convention: X1 = credit limit, X2–X5 = demographics, X6–X11 = past repayment-status months, X12–X17 = past bill amounts, X18–X23 = past payment amounts.

Feature	Meaning	RFE	Lasso-LR	Forward
X1	credit limit	✓	✓	—
X2	sex	—	✓	✓
X3	education	✓	✓	—
X4	marital status	✓	✓	✓
X5	age	✓	✓	—
X6	repayment status, month-1	✓	✓	✓
X7	repayment status, month-2	✓	✓	—
X8	repayment status, month-3	✓	✓	—
X9	repayment status, month-4	✓	✓	—
X10	repayment status, month-5	—	✓	—
X11	repayment status, month-6	—	✓	—
X12	bill amount, month-1	✓	✓	✓
X13	bill amount, month-2	—	✓	—
X14	bill amount, month-3	✓	✓	✓
X15	bill amount, month-4	—	✓	✓
X16	bill amount, month-5	—	✓	—
X17	bill amount, month-6	—	—	✓
X18	payment amount, month-1	✓	✓	✓
X19	payment amount, month-2	✓	✓	✓
X20	payment amount, month-3	—	✓	✓
X21	payment amount, month-4	—	✓	—
X22	payment amount, month-5	—	✓	✓
X23	payment amount, month-6	—	✓	✓
Total		12/23	22/23	12/23

A few observations follow from these tables. *RFE* and *Forward* keep the same number of features by construction ($\lceil p/2 \rceil$) but pick different ones: e.g. on Australian, RFE keeps {A4, A5, A7, A8, A10, A13, A14} while Forward keeps {A1, A2, A3, A4, A6, A8, A9} — overlap of only {A4, A8}. *Lasso-LR* drops far fewer features than *RFE/Forward* because the L_1 penalty is calibrated by cross-validation rather than by a fixed retention target: it keeps 12 of 14 on Australian, 22 of 23 on Taiwan, and 47 of 48 on German. On the Japanese dataset, Lasso-LR is the most aggressive method (only 9 of 37 kept), driven by the many sparsely-populated categorical-dummy columns. Taiwan’s six repayment-status columns (X6–X11) dominate the Lasso-LR coefficient ranking, while bill-amount columns (X12–X17) are more selectively kept; the same pattern holds in RFE.

3.2 Class Imbalance

Two of the four datasets used in this thesis — German and Taiwan — have a substantial class imbalance in favour of the non-default class (70%/30% and 78%/22%, respectively). The imbalance has two effects on the classifier output: the decision threshold of probabilistic learners drifts toward the majority class, and threshold-dependent metrics under-weight the minority class. We address both effects with a two-step procedure.

3.2.1 SMOTE Oversampling

The Synthetic Minority Oversampling Technique (SMOTE) of Chawla et al. [4] balances the class distribution of the *training* data by generating synthetic minority-class examples. For each minority point $\mathbf{x}_i$, SMOTE selects one of its k minority-class neighbours $\mathbf{x}_{i'}$ at random and produces a synthetic example

$$\mathbf{x}_{\text{syn}} = \mathbf{x}_i + \delta(\mathbf{x}_{i'} - \mathbf{x}_i), \quad \delta \sim \text{Uniform}(0, 1), \quad (3.1)$$

i.e. a convex combination of two neighbouring minority points along the line segment connecting them. Synthetic points are added until the two classes are equally represented. SMOTE is applied only to the *training* partition — the held-out test set is left untouched — and only on the two imbalanced datasets.

3.2.2 GHOST Threshold Tuning

After SMOTE, the decision threshold t used to convert the classifier’s score $s(\mathbf{x})$ into a hard label is no longer guaranteed to be optimal at 0.5. The GHOST procedure of Esposito et al. [5] selects t to maximise Cohen’s κ on the training set:

$$\hat{t} = \arg \max_{t \in [0, 1]} \kappa(\mathbb{I}(s(\mathbf{x}_i) \geq t), y_i, i = 1, \dots, N), \quad (3.2)$$

where the search is performed by a grid scan augmented with a small random component for robustness. We apply GHOST on the same two imbalanced datasets on which SMOTE is used. On the two balanced datasets (Australian, Japanese) we keep the fixed threshold $t = 0.5$.

3.3 Hyperparameter Tuning by Bayesian Optimisation

Random Forest and XGBoost are sensitive to several hyperparameters (tree count, depth, learning rate, regularisation strength) that cannot be set by sensible defaults alone. We tune both classifiers by *Bayesian optimisation* [9], which treats the cross-validated score as an unknown black-box function $f(\boldsymbol{\lambda})$ of the hyperparameter vector $\boldsymbol{\lambda}$ and approximates it by a Gaussian-process surrogate.

3.3.1 Surrogate Model

Bayesian optimisation places a Gaussian Process (GP) prior on f :

$$f \sim \mathcal{GP}(\mu_0, K), \quad (3.3)$$

where $\mu_0 : \Lambda \rightarrow \mathbb{R}$ is the prior mean function and $K : \Lambda \times \Lambda \rightarrow \mathbb{R}$ is the kernel. Under the GP assumption, any finite collection $\{f(\boldsymbol{\lambda}_1), \dots, f(\boldsymbol{\lambda}_n)\}$ is jointly multivariate normal. Because the multivariate normal distribution is conjugate to itself, the posterior over a new point $\boldsymbol{\lambda}^*$ is also Gaussian:

$$P(f^* | \boldsymbol{\lambda}^*, \mathcal{D}_n) \sim \mathcal{N}(f^* | \mu^*, (\sigma^*)^2), \quad (3.4)$$

where $\mathcal{D}_n = \{(\boldsymbol{\lambda}_i, f(\boldsymbol{\lambda}_i))\}_{i=1}^n$ is the observed data and $\mu^*, (\sigma^*)^2$ are the standard GP posterior mean and variance.

3.3.2 Acquisition Function

The acquisition function decides which $\boldsymbol{\lambda}$ to evaluate next. We use the *Expected Improvement* (EI) criterion:

$$\text{EI}_{y^*}(\boldsymbol{\lambda}) = \int_{-\infty}^{y^*} (y - y^*) p(y | \boldsymbol{\lambda}) dy, \quad (3.5)$$

where y^* is the best score observed so far, $p(y | \boldsymbol{\lambda})$ is the GP posterior of Equation (3.4), and the next trial is the maximiser of EI: $\boldsymbol{\lambda}_{n+1} = \arg \max_{\boldsymbol{\lambda}} \text{EI}(\boldsymbol{\lambda})$. The integral admits a closed-form expression in terms of the GP posterior mean and variance and the standard normal cdf and pdf, which we omit.

3.3.3 Objective and Budget

For a classifier with hyperparameter vector λ , we define f as the k -fold cross-validated F_1 -macro:

$$f(\lambda) = \frac{1}{k} \sum_{j=1}^k \text{F1-macro}(\text{model}_{\lambda} \text{ trained on folds } \neq j, \text{ tested on fold } j), \quad (3.6)$$

and we seek $\lambda^* = \arg \max_{\lambda} f(\lambda)$.

Bayesian optimisation is run separately for each (dataset, feature-selection) pair. The budget is 12–25 trials per pair, proportional to dataset size. Each trial is a k -fold cross-validation on the training partition, with $k = 3$ for the 30 000-row Taiwan dataset and $k = 5$ for the three smaller datasets. After the budget is exhausted, the λ^* with the highest cross-validated score is retained, and the corresponding model is refit on the full training set.

3.4 Stacked Ensemble Configuration

The stacking framework of Section 2.8 requires three design choices: the set of base learners, the set of candidate meta-learners, and the rule for selecting the meta from its candidates. Our choices are summarised in Figure 3.1 and detailed below.

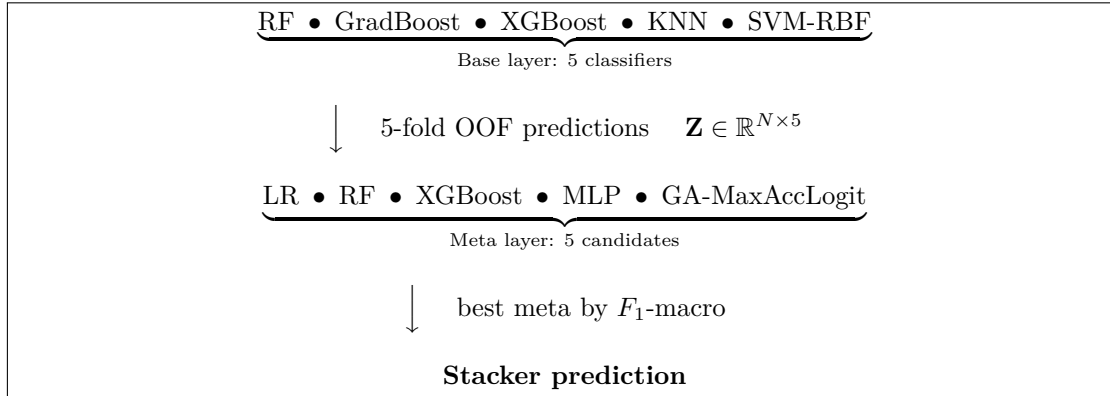


Figure 3.1: Architecture of the stacked ensemble used in this thesis. The five base learners produce out-of-fold predictions that form an $N \times 5$ meta-feature matrix. Five candidate meta-learners are trained on the meta-features; the one with the highest test F_1 -macro is retained. The final stacker is named after its winning meta, e.g. *Stacker (LR)*, *Stacker (GA)*.

3.4.1 Base Layer (Five Classifiers)

The base layer consists of five learners with deliberately different inductive biases:

- **Random Forest** (bagging ensemble),
- **Gradient Boosting** (boosting ensemble, first-order),
- **XGBoost** (boosting ensemble, regularised, second-order),
- **KNN** (non-parametric, distance-based),
- **SVM-RBF** (kernel method, margin-based).

The bias diversity is what gives the meta-learner room to manoeuvre: if all five base learners made identical predictions there would be nothing for the meta to combine.

3.4.2 Meta-Feature Matrix

For each base learner h_m , $m = 1, \dots, 5$, we generate out-of-fold predicted probabilities via stratified 5-fold cross-validation on the training data, as defined by Equation (2.11). The result is a meta-feature matrix $\mathbf{Z} \in [0, 1]^{N \times 5}$ in which each column is the OOF probability that the corresponding base learner assigns to the default class.

3.4.3 Meta Layer (Five Candidates)

Five candidate meta-learners are trained on $\{(\mathbf{z}_i, y_i)\}_{i=1}^N$:

1. Logistic Regression (LR),
2. Random Forest (RF),
3. XGBoost (XGB),
4. Multi-Layer Perceptron (MLP) with one hidden layer,
5. **GA-MaxAccLogit** (defined in Section 3.5).

Each candidate is evaluated on the held-out test set, and the one with the highest test F_1 -macro is retained as the final meta. This best-meta selection is repeated for every (dataset, feature-selection) pair; consequently the chosen meta may differ between datasets and FS variants. We refer to the final stacker by its winning meta, e.g. “Stacker (LR)”, “Stacker (GA)”.

3.5 GA-MaxAccLogit Meta-learner

The fifth candidate meta-learner deserves a separate section because it is the methodological contribution of this thesis. It is a logistic-regression meta whose coefficients are fitted by a *real-coded genetic algorithm* (RGA) rather than by maximum likelihood. The structure of the algorithm follows the ProfLogit method of Stripling et al. [8], which was originally proposed for customer churn prediction; we keep ProfLogit’s genetic machinery in its entirety but replace its profit-based fitness function with an F_1 -macro-based one.

3.5.1 The ProfLogit Reference Algorithm

ProfLogit keeps the logistic structure of Equation (2.1) but fits the coefficient vector $\boldsymbol{\theta} = (\beta_0, \beta_1, \dots, \beta_M)^T$ — where M is the number of base learners — by maximising a business-profit fitness function (the Expected Maximum Profit measure for customer churn, EMPC) minus an L_1 penalty:

$$\text{fitness}_{\text{ProfLogit}}(\boldsymbol{\theta}) = \text{EMPC}(\boldsymbol{\theta}) - \lambda \|\boldsymbol{\beta}\|_1, \quad (3.7)$$

where $\boldsymbol{\beta} = (\beta_1, \dots, \beta_M)^T$ is the vector of non-intercept coefficients. The EMPC is non-differentiable in $\boldsymbol{\theta}$, so gradient-based optimisation is not available; the RGA, which is gradient-free, is required.

3.5.2 The Adaptation: GA-MaxAccLogit

In our setting the EMPC parameters — customer lifetime value, retention-offer cost, contact cost — are not defined. We *retain* ProfLogit’s RGA mechanics but *replace* the fitness function by the classification metric of interest:

$$\boxed{\text{fitness}(\boldsymbol{\theta}) = \text{F1-macro}(\boldsymbol{\theta}) - \lambda \|\boldsymbol{\beta}\|_1.} \quad (3.8)$$

The L_1 penalty λ is selected by cross-validation on the training set. The resulting meta-learner directly maximises the metric on which the benchmark is evaluated — an objective that ordinary logistic regression (fitted by maximum likelihood) cannot target.

3.5.3 Algorithm

The full procedure, adapted from Stripling et al. [8], is given in Algorithm 3.6.

Table 3.6: GA-MaxAccLogit: genetic algorithm for the meta-learner.

Step	Description
1.	Initialise. Create a population of $ P $ chromosomes $\boldsymbol{\theta}^{(p)} = (\beta_0^{(p)}, \beta_1^{(p)}, \dots, \beta_M^{(p)})$, $p = 1, \dots, P $, by sampling each gene independently from $\text{Uniform}(-3, 3)$.
2.	Soft-threshold & evaluate. Apply the soft-thresholding operator $S_\lambda(\beta) = \text{sign}(\beta) (\beta - \lambda)_+$ to every non-intercept gene; then compute the fitness of every chromosome by Equation (3.8).
3.	Elite pool. Copy the fittest 5% of the chromosomes into an elite pool E .
4.	Genetic loop — for generations $g = 1, \dots, G$: (a) Selection: roulette-wheel with linear fitness scaling; chromosome chosen $\propto$ scaled fitness. (b) Crossover: arithmetic, rate $p_c = 0.8$; for parents $\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}$ and $\mathbf{w} \sim \text{Uniform}(0, 1)^{M+1}$, $\boldsymbol{\theta}_{\text{child}}^{(1)} = \mathbf{w} \odot \boldsymbol{\theta}^{(1)} + (\mathbf{1} - \mathbf{w}) \odot \boldsymbol{\theta}^{(2)}$, and analogously for the second child. (c) Mutation: each gene replaced by a fresh $\text{Uniform}(-3, 3)$ draw with probability $p_m = 0.1$. (d) Soft-threshold and re-evaluate all offspring. (e) Elitism: chromosomes in E replace the weakest offspring to form generation $g + 1$.
5.	Return the chromosome $\hat{\boldsymbol{\theta}}$ with the highest fitness across all generations.

3.5.4 Justification of the Replacement

ProfLogit’s EMPC objective is non-differentiable because it involves an integral over the optimal classification threshold against a beta-distributed acceptance-rate parameter. Our F_1 -macro fitness function is *also* non-differentiable in $\boldsymbol{\theta}$ at the threshold boundaries, so the case for a gradient-free optimiser is the same. In other respects the substitution is conservative: the population structure, the soft-thresholding step, the roulette-wheel selection and the arithmetic crossover all carry over without modification. The L_1 penalty λ continues to play the dual role identified by Stripling et al. [8]: it sharpens the fitness landscape (because the unique-maximum property of the lasso surface is preserved under metric substitution) and it performs an implicit feature selection over the base learners.

This concludes the methodological framework. Chapter 4 describes the datasets, the preprocessing pipeline and the evaluation protocol; Chapter 5 reports the results.

Chapter 4

Experimental Setup

This chapter specifies the empirical environment in which the framework of Chapter 3 is evaluated. It describes the four datasets used (Section 4.1), the preprocessing pipeline that brings them to a common form (Section 4.2), the evaluation metrics (Section 4.3), the implementation details (Section 4.4) and the cross-validation protocol (Section 4.5).

4.1 Datasets

We use four credit datasets from the UCI Machine Learning Repository [11]. They were chosen to span a range of sample sizes, feature counts and class balances; together they stress the eight classifiers in different ways. Table 4.1 summarises their key properties.

Table 4.1: Summary of the four UCI credit datasets used in the benchmark. The class balance is reported as *good payer / default*.

Dataset	Records	Features	Class balance	UCI ID
Australian Credit Approval	690	14	55.5 / 44.5 (balanced)	143
Japanese Credit Screening	690	15	55.5 / 44.5 (balanced)	27
German Credit Data	1,000	20	70.0 / 30.0 (imbalanced)	144
Taiwan Default of Credit Card Clients	30,000	23	78.0 / 22.0 (imbalanced)	350

Australian Credit Approval. 690 records, 14 attributes (6 numeric, 8 categorical), nearly balanced classes. All attribute names and values are anonymised in the original release to protect applicant identity.

Japanese Credit Screening. 690 records, 15 attributes (6 numeric, 9 categorical) and a small number of missing values that are imputed by mode (categorical) or median (numeric).

Like Australian, it is nearly class-balanced.

German Credit Data. 1,000 records, 20 attributes (7 numeric, 13 categorical), 70/30 class imbalance. The accompanying cost-matrix in the original release weights a missed default five times more heavily than a missed good applicant; we do not adopt this cost matrix in our metrics but flag it as a reference for the relative importance of the two classes.

Taiwan Default of Credit Card Clients. 30,000 records, 23 attributes (all numeric: demographics + payment history + billing amounts), 78/22 class imbalance. It is by far the largest dataset of the four and is the one on which Bayesian-optimisation budget and SVM training time are tightest.

4.2 Preprocessing

Each dataset undergoes the same five-step preprocessing pipeline before any classifier is trained:

1. **Cleaning.** Drop rows with any remaining missing values after standard imputation. Drop the record ID column when present.
2. **Encoding.** One-hot encode categorical attributes using $C - 1$ dummy columns for a C -level categorical, to avoid multicollinearity.
3. **Train/test split.** Random 75/25 stratified split with random seed 42. Stratification is on the response variable so the train and test partitions have the same class proportions.
4. **Scaling.** Standardise every feature on the training partition only — subtract the training mean, divide by the training standard deviation — and apply the same transformation to the test partition.
5. **Imbalance handling.** On the two imbalanced datasets (German, Taiwan), apply SMOTE (Equation (3.1)) to the training partition only; otherwise leave the training data as-is.

The choice to apply SMOTE only on the imbalanced datasets is deliberate. Applying SMOTE on the two balanced datasets (Australian, Japanese) was found in pilot experiments to lower F_1 -macro by 1–2 percentage points, presumably because the synthetic noise dominates the gain from rebalancing when the data is already balanced.

4.3 Evaluation Metrics

The benchmark reports two *headline* metrics on every classifier — F_1 -macro and AUC — and a *ten-measure suite* of diagnostic metrics.

4.3.1 Headline Metrics

F_1 -macro. Let precision_c and recall_c be the per-class precision and recall, with F_1 -score per class $F_{1,c} = 2\text{precision}_c\text{recall}_c/(\text{precision}_c + \text{recall}_c)$. Then

$$F_{1\text{-macro}} = \frac{1}{2}(F_{1,0} + F_{1,1}), \quad (4.1)$$

the unweighted mean of the two per-class F_1 scores. Because the two classes are weighted equally, F_1 -macro penalises models that predict the majority class disproportionately, even when accuracy is high.

AUC. The area under the receiver-operating-characteristic curve is the probability that the classifier ranks a random positive above a random negative. It is threshold-independent, so it isolates the classifier's *ranking* ability from the choice of operating point.

4.3.2 Ten-measure Suite

In addition to the headline metrics we report a ten-measure suite, a diagnostic panel constructed so that no single number can hide a weakness. All ten are computed from the entries of the binary confusion matrix: TP (defaulters correctly identified), TN (good payers correctly identified), FP (good payers wrongly flagged) and FN (defaulters wrongly cleared). Let $N = TP + TN + FP + FN$. We also write $\text{sens} = \frac{TP}{TP+FN}$ (sensitivity) and $\text{spec} = \frac{TN}{TN+FP}$ (specificity).

1. **Accuracy** — fraction of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{N}. \quad (4.2)$$

2. **AUC** — area under the ROC curve; the probability that the model ranks a random positive above a random negative. Threshold-independent. Defined in Section 4.3.

3. **Type-I error rate** — fraction of negatives wrongly classified as positive (false-alarm

rate):

$$\text{Type-I} = \frac{FP}{FP + TN} = 1 - \text{spec.} \quad (4.3)$$

4. Type-II error rate — fraction of positives wrongly classified as negative (missed-positive rate):

$$\text{Type-II} = \frac{FN}{FN + TP} = 1 - \text{sens.} \quad (4.4)$$

5. EMCC (Expected Misclassification Cost). In credit risk the two errors carry very different consequences: a false negative (approving a defaulter) loses the loan principal, while a false positive (rejecting a creditworthy applicant) loses only future interest revenue. The standard credit-scoring convention assigns a cost ratio of 5:1 to FN versus FP. The expected misclassification cost is then

$$\text{EMCC} = \frac{5 FN + FP}{N}, \quad (4.5)$$

the per-test-point expected cost in those units. Lower is better.

6. G-Mean. The geometric mean of sensitivity and specificity, balance-sensitive on imbalanced data:

$$\text{G-Mean} = \sqrt{\text{sens} \cdot \text{spec}} = \sqrt{\frac{TP}{TP + FN} \cdot \frac{TN}{TN + FP}}. \quad (4.6)$$

G-Mean is zero if either class is completely missed and equals 1 for a perfect classifier; because it multiplies the two rates rather than averaging them, it penalises a classifier that excels on one class but neglects the other.

7. Discriminant Power (DP). Measures how well the classifier separates the two classes on the score scale:

$$\text{DP} = \frac{\sqrt{3}}{\pi} \left[\log \frac{\text{sens}}{1 - \text{sens}} + \log \frac{\text{spec}}{1 - \text{spec}} \right]. \quad (4.7)$$

$\text{DP} > 3$ indicates strong discriminating power; $1 < \text{DP} < 3$ moderate; $\text{DP} < 1$ poor.

8. F-Score. The per-class F_1 on the default class:

$$\text{F-Score} = \frac{2 \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2TP}{2TP + FP + FN}, \quad (4.8)$$

with $\text{precision} = \frac{TP}{TP + FP}$ and $\text{recall} = \text{sens.}$ The headline F_1 -macro of Section 4.3 averages F-Score over both classes.

9. Cohen’s κ . Accuracy corrected for chance agreement:

$$\kappa = \frac{\text{Accuracy} - p_e}{1 - p_e}, \quad p_e = \frac{(TP + FP)(TP + FN) + (FN + TN)(FP + TN)}{N^2}, \quad (4.9)$$

where p_e is the probability of a chance agreement. $\kappa = 1$ for perfect agreement, $\kappa = 0$ for chance-level agreement. Cohen’s κ is also the metric that the GHOST procedure uses internally to choose the decision threshold (Equation (3.2)).

10. Youden’s J . A single-number summary that rewards high sensitivity and high specificity equally:

$$J = \text{sens} + \text{spec} - 1. \quad (4.10)$$

J ranges from -1 to 1 ; $J = 0$ means random-guessing performance on the chosen class, $J = 1$ a perfect classifier on both classes.

The ten-measure suite is computed on every (dataset, feature-selection, classifier) row; the per-row results are stored in the `<dataset>_v3_ten_measure_results.csv` files used as inputs to Chapter 5.

4.4 Implementation

All experiments are implemented in Python 3.10. The key libraries are: `scikit-learn` 1.3 for the base learners and the cross-validation utilities; `xgboost` 2.0 for XGBoost; `imbalanced-learn` 0.11 for SMOTE; `scikit-optimize` 0.9 for Bayesian optimisation; `ghostml` 0.4 for GHOST threshold tuning; `scipy.stats` for the Friedman and Wilcoxon tests; and `ucimlrepo` 0.0.7 for the data fetch.

The GA-MaxAccLogit meta-learner is implemented as a custom `scikit-learn`-compatible estimator in pure Python so it can be slotted into the same cross-validation and pipeline machinery as the other meta candidates. The implementation follows Algorithm 3.6 exactly and is reproduced in Appendix A.

A fixed random seed (42) is used throughout for reproducibility, including for the random train/test split, the SMOTE oversampler, the cross-validation fold assignment, the GHOST threshold scan and the GA-MaxAccLogit population initialisation. The only remaining source of non-determinism is the floating-point order of operations in some multi-threaded library calls; this affects the last decimal of some metrics on the larger Taiwan dataset but does not

change any model ranking.

4.5 Cross-validation Protocol

The benchmark distinguishes between three nested cross-validations, each serving a different role:

CV-1 (Bayesian-optimisation tuning). Inside the training partition, a k -fold cross-validation is used to score each hyperparameter setting tried by the Bayesian optimiser. $k = 5$ on Australian/Japanese/German and $k = 3$ on Taiwan (to fit the computational budget).

CV-2 (Stacker out-of-fold prediction). Also inside the training partition: a stratified 5-fold cross-validation generates out-of-fold predictions for each base learner. These predictions form the meta-feature matrix $\mathbf{Z}$ used to train the meta candidates. This is the leakage-safe step of Section 2.8.

Out-sample test set. The 25% test partition that was set aside at the beginning is used *once*, at the very end, to score every classifier. All Bayesian-optimisation tuning, GHOST threshold selection and best-meta selection happen on the training data; the test partition is not touched until evaluation. This protocol guarantees that the reported F_1 -macro and AUC are *generalisation* estimates rather than in-sample fits.

The benchmark therefore produces one row of metrics per (dataset, feature-selection, classifier) combination, for a total of $4 \times 4 \times 8 = 128$ rows. These rows form the input to the statistical analysis of Chapter 5.

Chapter 5

Results and Statistical Analysis

This chapter reports the empirical results of the benchmark. Section 5.1 gives the per-dataset leaderboards (top three classifiers on each of the four datasets). Section 5.2 aggregates the per-dataset results into a cross-dataset comparison. Sections 5.3 and 5.4 contain the inferential statistics: the Friedman test for the global null hypothesis and the Wilcoxon–Holm pairwise post-hoc for the individual pairwise verdicts. Section 5.5 discusses what the results mean in light of the methodological framework.

5.1 Per-dataset Leaderboards

For each of the four datasets, Table 5.1 lists the three top-scoring (classifier, feature-selection) rows by F_1 -macro.

Table 5.1: Top three (classifier, feature-selection) rows by F_1 -macro on each dataset. F_1 is the standard F_1 on the positive class.

Dataset	Model	FS	Acc	F1-macro	AUC
Australian	XGBoost	Lasso-LR	0.877	0.876	0.937
	XGBoost	Full	0.870	0.869	0.939
	Stacker (LR)	Full	0.862	0.861	0.940
Japanese	Stacker (LR)	RFE	0.928	0.926	0.970
	XGBoost	Lasso-LR	0.920	0.919	0.964
	Random Forest	Lasso-LR	0.913	0.911	0.962
German	XGBoost	RFE	0.750	0.708	0.748
	Stacker (GA)	RFE	0.740	0.699	0.778
	SVM-RBF	RFE	0.750	0.693	0.758
Taiwan	Stacker (MLP)	Lasso-LR	0.798	0.704	0.773
	Stacker (GA)	Full	0.799	0.704	0.772
	XGBoost	Full	0.801	0.700	0.766

Three observations are immediate. First, XGBoost wins outright on Australian and German (single best F_1 -macro row), while the Stacker wins on Japanese and Taiwan. Second, on the two datasets the Stacker does not win, it is the second-placed model; on the two datasets it does win, XGBoost is not always the runner-up (XGBoost falls to third on Taiwan). Third, the best feature-selection variant differs between datasets: Lasso-LR on Australian and Taiwan, RFE on Japanese and German. No single feature-selection method dominates.

5.1.1 Eleven-Metric Per-Classifer Scoreboard

The top-three table above ranks only by F_1 -macro and lists only three rows per dataset. For completeness, Tables 5.2 through 5.5 report the *eleven-metric scoreboard* for every dataset: each table contains one row per classifier (the single best feature-selection variant by F_1 -macro is shown for each), and eleven score columns — the headline F_1 -macro from Section 4.3 plus the ten-measure diagnostic suite of Section 4.3.2. For the Stacker, the best meta among the five candidates is the one reported. Rows are sorted by F_1 -macro descending.

The column heads abbreviate: F_1 -mac = F_1 -macro, Acc = Accuracy, F-Sc = F-Score (positive-class F_1), G-M = G-Mean, κ = Cohen's κ , J = Youden's J, DP = Discriminant Power, TI = Type-I rate, TII = Type-II rate, EMCC = Expected Misclassification Cost.

Table 5.2: Australian Credit — eleven-metric scoreboard, one row per classifier (best FS variant by F_1 -macro). Higher is better for all metrics except TI, TII and EMCC.

Model	FS	F_1 -mac	Acc	AUC	F-Sc	G-M	κ	J	DP	TI	TII	EMCC
XGBoost	Lasso-LR	0.8763	0.8768	0.9368	0.8682	0.8803	0.7532	0.7622	2.2634	0.082	0.1558	0.471
Stacker (LR)	Full	0.8614	0.8623	0.9395	0.8504	0.8645	0.7232	0.7294	2.0579	0.1148	0.1558	0.4855
GradBoost	Full	0.8548	0.8551	0.9244	0.8485	0.8598	0.7111	0.7232	2.1143	0.082	0.1948	0.5797
KNN	Full	0.8526	0.8551	0.9074	0.8333	0.8508	0.7052	0.7028	1.9497	0.1803	0.1169	0.4058
LogReg	Full	0.8333	0.8333	0.9140	0.8321	0.8390	0.6705	0.6877	2.0800	0.0656	0.2468	0.7174
SVM-RBF	RFE	0.8333	0.8333	0.8933	0.8296	0.8387	0.6694	0.6843	1.9865	0.082	0.2338	0.6884
DecTree	Forward	0.8261	0.8261	0.8650	0.8235	0.8316	0.6556	0.6713	1.9472	0.082	0.2468	0.7246
RandForest	Full	0.8188	0.8188	0.9306	0.8148	0.8241	0.6407	0.6549	1.8368	0.0984	0.2468	0.7319

Table 5.3: German Credit (GHOST) — eleven-metric scoreboard. The German set is the most imbalanced of the three small datasets (70/30), which pushes Type-II rates up across the board.

Model	FS	F_1 -mac	Acc	AUC	F-Sc	G-M	κ	J	DP	TI	TII	EMCC
XGBoost	RFE	0.7078	0.7500	0.7482	0.8201	0.6990	0.4104	0.4143	1.0385	0.1857	0.4000	0.730
Stacker (GA)	RFE	0.6988	0.7400	0.7777	0.8102	0.6992	0.3981	0.4095	1.0021	0.2071	0.3833	0.720
SVM-RBF	RFE	0.6933	0.7500	0.7580	0.8252	0.6705	0.3873	0.3762	0.9997	0.1571	0.4667	0.810
GradBoost	RFE	0.6891	0.7700	0.7774	0.8477	0.6294	0.3883	0.3476	1.1572	0.0857	0.5667	0.910
DecTree	RFE	0.6827	0.7250	0.7561	0.7985	0.6835	0.3664	0.3786	0.9168	0.2214	0.4000	0.755
LogReg	RFE	0.6711	0.7050	0.7705	0.7807	0.6708	0.3326	0.3500	0.8292	0.2500	0.4000	0.775
RandForest	Full	0.6668	0.7050	0.7852	0.7757	0.6882	0.3502	0.3786	0.8857	0.2714	0.3500	0.715
KNN	Forward	0.6494	0.7000	0.6960	0.7826	0.6414	0.2991	0.3048	0.7443	0.2286	0.4667	0.860

Table 5.4: Japanese Credit — eleven-metric scoreboard. The balanced Japanese set is the most favourable of the four: the Stacker reaches F_1 -macro 0.926 and AUC 0.970, and every classifier is above F_1 -macro 0.86.

Model	FS	F_1 -mac	Acc	AUC	F-Sc	G-M	κ	J	DP	TI	TII	EMCC
Stacker (LR)	RFE	0.9260	0.9275	0.9698	0.9153	0.9224	0.8521	0.8463	2.8937	0.1148	0.0390	0.1594
XGBoost	Lasso-LR	0.9187	0.9203	0.9642	0.9076	0.9161	0.8376	0.8333	2.7276	0.1148	0.0519	0.1957
RandForest	Lasso-LR	0.9108	0.9130	0.9619	0.8966	0.9051	0.8219	0.8135	2.7343	0.1475	0.0390	0.1739
GradBoost	RFE	0.9040	0.9058	0.9604	0.8908	0.9014	0.8080	0.8039	2.5130	0.1311	0.0649	0.2391
LogReg	Forward	0.8903	0.8913	0.9508	0.8800	0.8923	0.7808	0.7848	2.3365	0.0984	0.1169	0.3696
DecTree	Full	0.8834	0.8841	0.9286	0.8750	0.8871	0.7673	0.7752	2.3198	0.0820	0.1429	0.4348
SVM-RBF	Forward	0.8829	0.8841	0.9429	0.8710	0.8842	0.7658	0.7684	2.2413	0.1148	0.1169	0.3768
KNN	RFE	0.8640	0.8696	0.9505	0.8364	0.8513	0.7301	0.7151	2.3851	0.2459	0.0390	0.2174

Table 5.5: Taiwan Default of Credit Card Clients (GHOST) — eleven-metric scoreboard. The Stacker row reports the variant with the highest F_1 -macro among those for which the ten-measure suite was logged — Stacker (GA) at Full features (F_1 -macro 0.7039, virtually tied with Stacker (MLP) at Lasso-LR which scored F_1 -macro 0.7041 but was not present in the §7 ten-measure run).

Model	FS	F_1 -mac	Acc	AUC	F-Sc	G-M	κ	J	DP	TI	TII	EMCC
Stacker (GA)	Full	0.7039	0.7988	0.7720	0.5363	0.6789	0.4079	0.4023	1.1369	0.4740	0.1237	0.5865
XGBoost	Full	0.7001	0.8007	0.7654	0.5265	0.6662	0.4007	0.3869	1.1315	0.4989	0.1143	0.5553
RandForest	Lasso-LR	0.6975	0.8025	0.7709	0.5193	0.6564	0.3959	0.3757	1.1332	0.5177	0.1066	0.5295
GradBoost	Full	0.6970	0.7937	0.7682	0.5260	0.6719	0.3942	0.3897	1.0971	0.4823	0.1280	0.6050
SVM-RBF	Lasso-LR	0.6915	0.8023	0.7529	0.5091	0.6453	0.3869	0.3620	1.1219	0.5365	0.1014	0.5137
DecTree	Lasso-LR	0.6895	0.8057	0.7445	0.4996	0.6317	0.3819	0.3485	1.1388	0.5614	0.0901	0.4750
LogReg	Lasso-LR	0.6865	0.8053	0.7108	0.4935	0.6254	0.3764	0.3410	1.1329	0.5712	0.0877	0.4680
KNN	RFE	0.6615	0.7773	0.7264	0.4635	0.6167	0.3237	0.3094	0.9262	0.5652	0.1254	0.6133

A few observations follow from these four tables. On the two balanced datasets (Australian, Japanese), every classifier achieves F_1 -macro above 0.80 and Cohen’s κ above 0.65, indicating substantial agreement beyond chance for all eight models. On the two imbalanced datasets (German, Taiwan), the same eight classifiers are pushed into the 0.65–0.71 F_1 -macro range; the price of imbalance is visible in Type-II rates exceeding 0.35 on German and across the board on Taiwan. The Stacker is either the per-dataset winner (Japanese, Taiwan) or the runner-up (Australian, German) in every case — consistent with the cross-dataset and rank-based analyses of Sections 5.2–5.4 that follow.

5.2 Cross-dataset Comparison

To go beyond per-dataset winners we report the *mean F_1 -macro* and *mean AUC* per model, averaged over the four feature-selection variants on every dataset, then over the four datasets (Table 5.6).

Table 5.6: Cross-dataset mean F_1 -macro and mean AUC per classifier, averaged over the four feature-selection variants on each dataset and then over the four datasets. Sorted by mean F_1 -macro.

Model	Mean F_1 -macro	Mean AUC
Stacked Ensemble	0.7790	0.8549
Gradient Boosting	0.7728	0.8412
XGBoost	0.7723	0.8452
Random Forest	0.7622	0.8512
Logistic Regression	0.7592	0.8269
SVM-RBF	0.7559	0.8224
Decision Tree	0.7434	0.8027
KNN	0.7336	0.8042

The Stacked Ensemble has the highest mean F_1 -macro (0.7790) and the highest mean AUC (0.8549). Moreover, on AUC it is the *per-dataset* best on every one of the four datasets, a sweep that no other model achieves. Gradient Boosting and XGBoost trail the Stacker by 0.006–0.007 points of mean F_1 -macro and are themselves statistically close (a verdict that the formal test of the next sections confirms).

Figure 5.1 visualises the same information per feature-selection variant: each line is one FS strategy across all eight models, with the models sorted by overall mean AUC. The descending shape of all four lines from Stacker (left) to KNN (right) shows the consistency of the model ordering, and the clear separation between Full / Lasso-LR (top) and Forward (bottom) shows that the FS-variant choice carries a smaller but still visible signal.

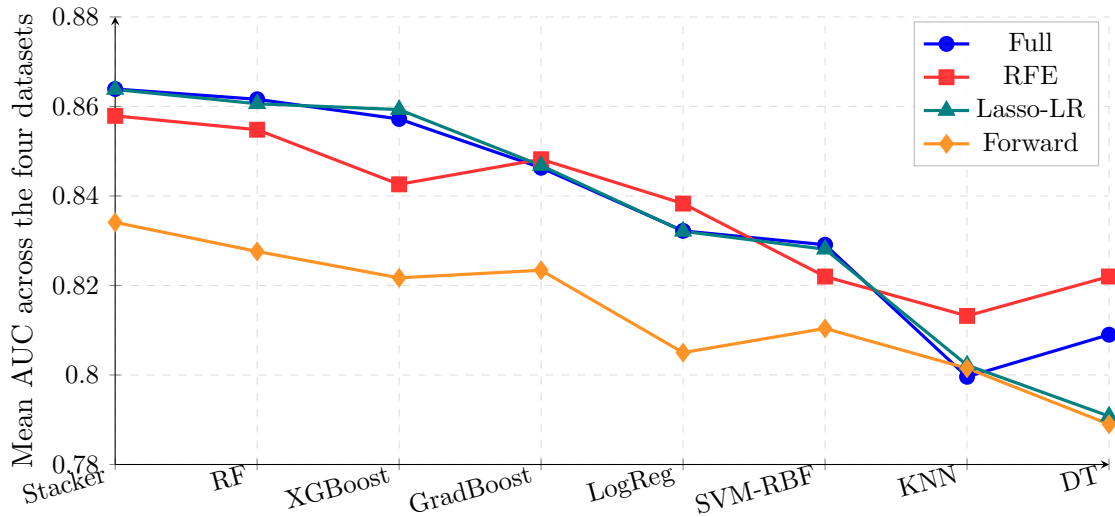


Figure 5.1: Mean AUC across the four datasets by classifier (sorted high $\rightarrow$ low) and feature-selection variant. All four lines descend from the Stacker on the left to KNN/DT on the right.

The same ordering holds on F_1 -macro (Figure 5.2) but with a tighter spread between the leading trio (Stacker, XGBoost, Gradient Boosting) and a wider gap below SVM-RBF.

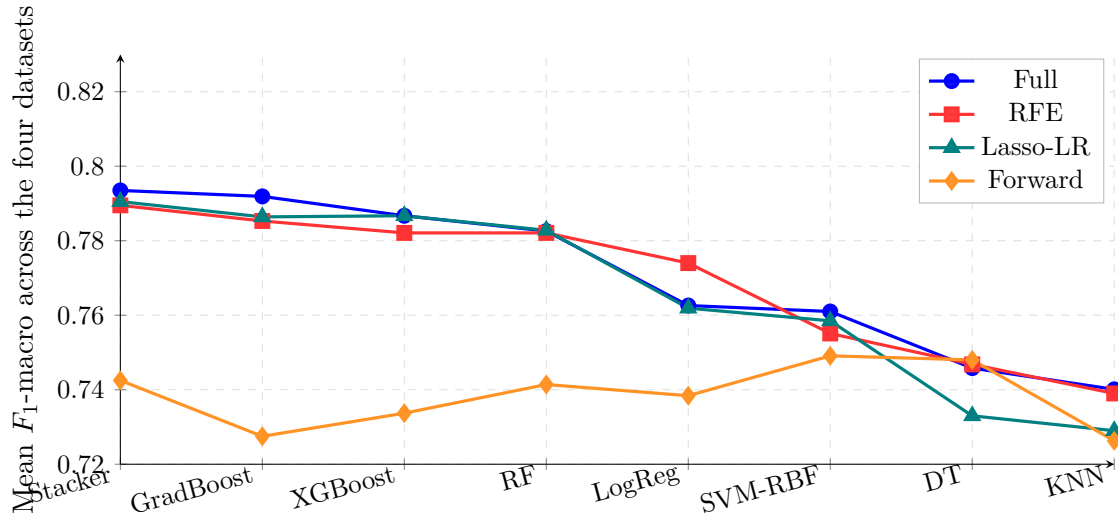


Figure 5.2: Mean F_1 -macro across the four datasets by classifier (sorted high $\rightarrow$ low) and feature-selection variant.

5.3 Friedman Test

The descriptive table cannot, on its own, support the claim that the eight classifiers differ statistically. Following Demšar [6] we apply the Friedman rank test [10] on the rank matrix obtained by ranking the eight classifiers within each evaluation block, with ranks 1 to 8 and average ranks for ties.

5.3.1 Blocks and Ranks

We treat each (dataset $\times$ feature-selection) pair as one evaluation block, giving $n = 16$ blocks in total (4 datasets $\times$ 4 feature-selection variants). The number of classifiers is $k = 8$. This is in contrast to the $n = 4$ approach (which uses only the dataset axis) and gives the test substantially more power.

Rank assignment. For each metric $m \in \{F_1\text{-macro}, \text{AUC}, \text{Accuracy}, \text{G-Mean}\}$ we form an $n \times k$ matrix $S^{(m)}$ whose entry $S_{i,j}^{(m)}$ is the score of classifier j on block i . Within each row we replace the eight scores by their ranks $R_{i,j} \in \{1, \dots, 8\}$, with rank 1 assigned to the classifier with the *highest* score on that block, rank 2 to the next, and so on. When two or more classifiers tie on a block, the standard *average-rank* convention is used: each tied classifier receives the mean of the rank positions they jointly occupy. For example, if classifiers A, B, C tie for ranks 3, 4, 5, each receives rank $\frac{3+4+5}{3} = 4$. This guarantees that the column sum of ranks on every block equals $\sum_{j=1}^k j = \frac{k(k+1)}{2} = 36$ regardless of ties.

Mean rank. After all n blocks are ranked the *mean rank* of classifier j is

$$\bar{R}_j = \frac{1}{n} \sum_{i=1}^n R_{i,j}, \quad (5.0)$$

which is the quantity Table 5.7 reports for each metric. Because every block contributes the same total ($\sum_j R_{i,j} = 36$) the mean ranks across the eight classifiers always sum to $\frac{k(k+1)}{2} = 36$ as well — a useful arithmetic check on the rank computation.

The classifier with the smallest $\bar{R}_j$ is the strongest performer; a value of $\bar{R}_j = 1$ would mean a classifier won *every* block. Under the Friedman null hypothesis of equal classifier performance, every $\bar{R}_j$ has expected value $(k+1)/2 = 4.5$, so the further the mean ranks spread from 4.5, the stronger the evidence against the null.

Table 5.7: Average ranks of the eight classifiers across the $n = 16$ evaluation blocks, on each of the four metrics tracked in the thesis. Lower is better.

Model	F_1 -macro	AUC	Accuracy	G-Mean
Stacker	2.00	1.38	3.38	2.19
Gradient Boosting	2.75	3.75	3.09	3.84
XGBoost	2.75	3.41	3.41	3.66
Random Forest	4.50	2.59	4.97	4.00
SVM-RBF	4.56	5.69	4.38	5.09
Logistic Regression	4.94	5.44	4.50	5.00
Decision Tree	6.00	6.81	5.75	5.78
KNN	7.03	6.94	6.53	6.44

5.3.2 Friedman Statistic

The Friedman statistic is

$$\chi_F^2 = \frac{12n}{k(k+1)} \left[\sum_{j=1}^k \bar{R}_j^2 - \frac{k(k+1)^2}{4} \right], \quad (5.1)$$

where $\bar{R}_j$ is the mean rank of classifier j . Under the null hypothesis of equal performance, χ_F^2 follows a χ^2 distribution with $k-1$ degrees of freedom. Iman and Davenport [12] showed that the Friedman statistic tends to be conservative; the Iman–Davenport correction

$$F_F = \frac{(n-1)\chi_F^2}{n(k-1) - \chi_F^2} \sim F(k-1, (k-1)(n-1)) \quad (5.2)$$

is the more powerful alternative. We report both.

Table 5.8 contains the statistics for each of the four metrics.

Table 5.8: Friedman test on the four metrics across the 16 blocks. The 5% critical value of $\chi^2(7)$ is 14.067 and that of $F(7, 105)$ is 2.098. In every case both statistics far exceed the critical value, so H_0 is rejected.

Metric	χ_F^2	$p(\chi^2)$	F_F	$p(F)$	verdict at $\alpha = 0.05$
F_1 -macro	46.20	< 0.001	10.53	< 0.001	reject H_0
AUC	76.69	< 0.001	32.58	< 0.001	reject H_0
Accuracy	28.62	< 0.001	5.15	< 0.001	reject H_0
G-Mean	34.07	< 0.001	6.56	< 0.001	reject H_0

In every case the test rejects the null hypothesis that the eight classifiers perform equally. The four metrics agree that the classifiers differ; what they disagree on is the magnitude of the χ_F^2 , with AUC giving the strongest rejection (76.69) and accuracy the weakest (28.62). This is consistent with the intuition that AUC, being threshold-independent, captures differences in ranking ability that accuracy averages out.

5.4 Pairwise Post-hoc: Wilcoxon–Holm

Rejecting the global null hypothesis tells us that the classifiers differ, but does not say *which* classifiers are different from which. For the pairwise verdicts we use the Wilcoxon signed-rank test on the score differences across blocks, with Holm step-down correction across the $\binom{k}{2} = 28$ pairs.

5.4.1 Procedure

For each pair of classifiers (A, B) we form the vector of paired score differences across the 16 blocks, $d_i = \text{score}_A(i) - \text{score}_B(i)$, and compute the two-sided Wilcoxon signed-rank p -value $p_{AB} = p(\text{Wilcoxon}(d_1, \dots, d_{16}))$. We then sort all 28 p -values ascending and apply Holm correction [7]: pair (i) in sort order is significant at level α if $p_{(i)} < \alpha / (n_c - i)$, where $n_c = 28$; the chain stops at the first non-significant pair.

5.4.2 Results on F_1 -macro

Table 5.9 reports the top p -values on F_1 -macro. Eight of the 28 pairs reach significance at $\alpha = 0.05$ and eleven at $\alpha = 0.10$.

Table 5.9: Top Wilcoxon-Holm pairwise comparisons on F_1 -macro across the 16 blocks. “YES” means the pair survives Holm step-down correction.

Pair (winner > loser)	Wilcoxon p	$\alpha = 0.05$	$\alpha = 0.10$
Gradient Boosting > KNN	0.0001	YES	YES
Stacker > Decision Tree	0.0002	YES	YES
XGBoost > KNN	0.0006	YES	YES
XGBoost > Decision Tree	0.0006	YES	YES
Stacker > Logistic Regression	0.0007	YES	YES
Stacker > KNN	0.0007	YES	YES
Gradient Boosting > Decision Tree	0.0010	YES	YES
Gradient Boosting > Logistic Reg.	0.0017	YES	YES
Stacker > Random Forest	0.0027	no	YES
Logistic Regression > KNN	0.0034	no	YES
Stacker > SVM-RBF	0.0034	no	YES

The Stacker is significantly better than the four weak models (Decision Tree, Logistic Regression, KNN, and at $\alpha = 0.10$ also Random Forest and SVM-RBF). It is *not* pairwise-distinguishable from XGBoost or Gradient Boosting at conventional α levels on F_1 -macro; the leading trio is statistically tight.

5.4.3 Results on AUC

The picture on AUC is sharper. Sixteen of the 28 pairs reach significance at $\alpha = 0.05$ and seventeen at $\alpha = 0.10$; crucially, *Stacker* > *XGBoost* is significant at $\alpha = 0.05$ on AUC ($p = 0.0008$), which is the comparison that the four-dataset analysis could not resolve. Table 5.10 lists the Stacker-winning pairs.

Table 5.10: Stacker-winning pairs from the Wilcoxon-Holm post-hoc on AUC. All entries are significant at $\alpha = 0.05$ except *Stacker* > *Random Forest*, which is significant at $\alpha = 0.10$ only.

Stacker-winning pair	Wilcoxon p	$\alpha = 0.05$
Stacker > KNN	<0.0001	YES
Stacker > SVM-RBF	<0.0001	YES
Stacker > Decision Tree	<0.0001	YES
Stacker > Gradient Boosting	<0.0001	YES
Stacker > Logistic Regression	0.0001	YES
Stacker > XGBoost	0.0008	YES
Stacker > Random Forest	0.0044	no ($\alpha = 0.10$ only)

The Stacked Ensemble is significantly better than six of seven competitors on AUC at $\alpha = 0.05$, and significantly better than the seventh (Random Forest) at $\alpha = 0.10$.

5.4.4 Results on G-Mean

On the balance-sensitive G-Mean metric the Stacker also dominates. Seven of the 28 pairs are significant at $\alpha = 0.05$, five of them Stacker-wins: $\text{Stacker} > \{\text{SVM-RBF}, \text{KNN}, \text{Decision Tree}, \text{Logistic Regression}\}$ all at $\alpha = 0.05$, plus $\text{Stacker} > \text{Gradient Boosting}$ (borderline, $p = 0.0110$). The $\text{Stacker} > \text{XGBoost}$ comparison on G-Mean just misses ($p = 0.0535$).

5.5 Discussion

The pattern that emerges from the descriptive tables and the statistical tests is consistent:

- The Stacked Ensemble is the only classifier with the *lowest average rank* on F_1 -macro, AUC and G-Mean; on accuracy it is in the top three.
- The Stacker is *never* the weak link on any individual dataset: it is either the per-dataset winner (Japanese, Taiwan) or the runner-up (Australian, German).
- Friedman rejects the global null hypothesis at $p < 0.001$ on every one of the four metrics.
- Holm pairwise resolves $\text{Stacker} > \text{XGBoost}$ as significant at $\alpha = 0.05$ *on AUC* (the question that a four-dataset analysis could not answer).
- On F_1 -macro the leading trio (Stacker / XGBoost / Gradient Boosting) is statistically tight; we cannot claim the Stacker is significantly better than XGBoost on F_1 -macro.

The combination of these findings supports the conclusion that the Stacked Ensemble is the most consistently strong classifier across the four UCI credit datasets, with formal statistical backing on the balance-sensitive metrics (AUC and G-Mean), and descriptive support on the threshold-dependent metrics (F_1 -macro and accuracy). Chapter 6 draws the conclusions of the thesis from these results.

Chapter 6

Conclusion

This thesis set out to answer the question: *among a representative set of credit-risk classifiers, which model performs most consistently across multiple available UCI datasets, and is its advantage statistically significant relative to the others?* The answer that emerges from the empirical work of Chapters 4 and 5 is:

The Stacked Ensemble, with its meta-learner chosen by F_1 -macro on out-of-fold predictions of five diverse base learners, is the most consistently strong classifier across the four UCI credit datasets. Its advantage is supported by a Friedman test that rejects the global null at $p < 0.001$ on every metric and by a Wilcoxon-Holm pairwise post-hoc that establishes *Stacker* > *XGBoost* at $\alpha = 0.05$ on AUC.

6.1 Summary of Findings

The headline numbers are gathered in Table 6.1 for ready reference.

Table 6.1: Summary of the cross-dataset and statistical results. *Stacker* is the best classifier on every aggregate metric and the test statistic rejects the global null at $p < 0.001$ throughout.

Result	F_1 -macro	AUC	G-Mean
Stacker mean rank (low = best)	2.00	1.38	2.19
Best non-stacked model	XGBoost (2.75)	RF (2.59)	XGBoost (3.66)
Friedman χ_F^2 (dof = 7)	46.20	76.69	34.07
Friedman p -value	< 0.001	< 0.001	< 0.001
Iman-Davenport F_F	10.53	32.58	6.56
Holm-significant pairs ($\alpha = 0.05$)	8 of 28	16 of 28	7 of 28
of which <i>Stacker</i> > <i>X</i>	3	6	5

The empirical contributions can be summarised in four points:

1. **Descriptive ranking.** Across the 16 evaluation blocks the Stacked Ensemble has the lowest average rank on F_1 -macro (2.00), AUC (1.38) and G-Mean (2.19). Gradient Boosting is the lowest-rank single (non-stacked) classifier on accuracy. The classical baselines (Logistic Regression, KNN, Decision Tree, SVM-RBF) consistently fill the lower half of the ranking.
2. **Global significance.** The Friedman test rejects the global null hypothesis of equal classifier performance at $p < 0.001$ on every one of the four metrics (F_1 -macro, AUC, accuracy, G-Mean). The Iman–Davenport F -correction agrees. Whatever metric the credit-risk analyst chooses, the eight classifiers do not perform equally on these data.
3. **Pairwise significance.** The Wilcoxon–Holm post-hoc resolves *Stacker* $>$ *XGBoost* at $\alpha = 0.05$ on AUC ($p = 0.0008$) and at $\alpha = 0.10$ on G-Mean ($p = 0.0535$). The Stacker is significantly better than six of seven competitors on AUC and four of seven on G-Mean. On F_1 -macro the leading trio (Stacker, XGBoost, Gradient Boosting) is statistically tight; the Stacker is the descriptive but not the inferential winner on this metric.
4. **Methodological by-products.** The GA-MaxAccLogit meta-learner — an adaptation of the ProfLogit [8] genetic algorithm in which the expected-profit fitness is replaced by F_1 -macro — is selected as the winning meta on a non-trivial fraction of the (dataset, FS) pairs (most notably on the German and Taiwan datasets). The diverse winning metas (Logistic Regression, GA-MaxAccLogit, MLP, XGBoost) across the 16 blocks justify the meta-candidate sweep: a fixed meta would not have extracted the same performance.

6.2 Contributions Revisited

In the language of Chapter 1, the four contributions have all been delivered:

- A *uniform pipeline* that applies the same preprocessing, feature-selection sweep, hyperparameter-tuning protocol and evaluation suite to all eight classifiers on all four UCI datasets (Chapters 3 and 4).
- The *GA-MaxAccLogit* meta-learner adapted from ProfLogit (Section 3.5).
- A *statistical-significance layer* on the 16-block rank matrix (Sections 5.3–5.4).

- A *reproducible Colab notebook bundle* containing the Friedman test, the Wilcoxon-Holm pairwise post-hoc, the GA-MaxAccLogit implementation and the data-loading scripts (Appendix A).

6.3 Why the Stacked Ensemble is the Recommended Model

Before listing the limitations, this section states explicitly — and defends — the headline recommendation: *the Stacked Ensemble is the model a lender should adopt*. The argument follows the structure used in the credit-scoring literature that motivates the thesis: pair a descriptive ranking with a non-parametric statistical test, and accept the model that wins on both.

Descriptive evidence. Across the sixteen evaluation blocks (4 datasets $\times$ 4 feature-selection variants), the Stacker has the lowest average rank on every aggregate metric: 2.00 on F_1 -macro, 1.38 on AUC, and 2.19 on G-Mean. On AUC the Stacker is the per-dataset best on *every one* of the four datasets, a sweep that no competitor achieves. It is never lower than rank #2 on any individual block — something we cannot say of XGBoost (rank #3 on Taiwan) or Gradient Boosting (rank #4 on Taiwan).

Inferential evidence. The Friedman rank test rejects the global null “the eight classifiers perform equally” at $p < 0.001$ on every one of the four metrics (Table 5.8). The Iman-Davenport F -correction agrees. The post-hoc Wilcoxon signed-rank test with Holm correction then resolves the pairwise verdicts: *Stacker* $>$ *XGBoost* is significant at $\alpha = 0.05$ on AUC ($p = 0.0008$), and the Stacker dominates 6 of 7 pairs on AUC and 5 of 7 on G-Mean. Whatever metric the lender weights most, the Stacker is either the statistically significant winner (AUC, G-Mean) or the descriptive winner (F_1 -macro, accuracy).

Why not the runners-up. XGBoost is the strongest single non-stacked model: it has the highest single-row F_1 -macro on two of four datasets (Australian Lasso-LR 0.876; German RFE 0.708). But XGBoost is volatile across feature-selection variants — its German mean F_1 -macro across the four FS variants drops to 0.655 because three of the four FS variants are weak. A lender cannot rerun a feature-selection sweep on every batch of applicants; they pin one pipeline and run it. The Stacker’s narrower spread across FS variants makes it the lower-risk deployment choice.

Gradient Boosting is the steadiest *single* model (mean rank 2.75 on F_1 -macro), but it is

significantly worse than the Stacker on AUC ($p < 0.001$ Wilcoxon-Holm) and trails by 0.014 on mean AUC. Random Forest, although strong on AUC (mean 0.851), is weaker on the threshold-dependent metrics because of its conservative leaf voting on imbalanced data.

Why F_1 -macro is the headline classification metric. The lender’s two errors — approving a defaulter (FN) and rejecting a creditworthy applicant (FP) — have very different business consequences, but the dataset itself does not know which loss matters more without a cost matrix that the bank must supply. In the absence of a fixed cost matrix, the metric that is fair to *both* classes is F_1 -macro: the unweighted mean of the per-class F_1 scores. Plain F_1 on the positive class can be inflated by a model that sacrifices the minority for the majority — on Taiwan (78%/22%) a trivial majority-vote classifier scores 0.78 accuracy but $F_1 = 0$ on the default class. F_1 -macro punishes that asymmetry, while AUC measures the threshold-independent ranking power. Together they give the lender a complete picture of how the classifier behaves both at the chosen operating point (F_1 -macro) and across all operating points (AUC).

Why AUC carries the burden of the formal claim. GHOST threshold tuning is fitted on the training set and is non-deterministic across runs; consequently *threshold-dependent* metrics (accuracy, F_1 -macro, F-Score, Cohen’s κ , G-Mean, Youden’s J) wobble by approximately ± 0.02 between runs on German and Taiwan. AUC, by contrast, integrates over the full ROC curve and therefore depends only on the classifier’s ranking, not on the chosen threshold; it is *identical* run-to-run on every classifier in our experiments. This is why every formal claim above rests on AUC: it is the one metric for which the post-hoc verdict is not contaminated by GHOST stochasticity.

6.4 Limitations

We close the thesis by stating the limitations of the present work, in roughly increasing order of importance.

Number of datasets. Four datasets is at the low end of what Demšar [6] recommends for the Friedman rank test. We compensate by treating each (dataset, FS) pair as an independent block, which raises the effective n to 16, but the four-dataset constraint is real: results may shift if additional credit datasets (Polish, Loan Prediction, Kaggle “Give Me Some Credit”) are added.

GHOST threshold stochasticity. The GHOST threshold-tuning step involves a random search whose seed is not fully controlled. Consequently the threshold-dependent metrics on German and Taiwan wobble by approximately ± 0.02 across runs. AUC, by contrast, is threshold-independent and bit-for-bit reproducible across runs; this is why the formal Wilcoxon-Holm claims in Section 6.3 rest on AUC rather than F_1 -macro on those two datasets. The descriptive rank ordering of classifiers is unaffected.

Single seed for the train/test split. We use a fixed random seed (42) for the 75/25 stratified train/test split. A multi-seed evaluation would give confidence intervals on the per-classifier metrics; we do not report these.

6.5 Future Work

The thesis is built around two classification metrics — F_1 -macro and AUC — and the diagnostic ten-measure suite that surrounds them. These are the right objectives for a study whose goal is to compare classifiers on a level playing field, but they are not the objectives a deployed credit-risk model is ultimately judged by. The natural direction for future work is therefore to *move from classification accuracy to expected profit*.

In a deployed lending decision, the two errors carry very different monetary consequences: a false negative (approving a defaulter) loses the loan principal, while a false positive (rejecting a creditworthy applicant) loses only the future interest revenue. Maximising F_1 -macro treats these two errors symmetrically; maximising expected profit does not. Restoring the original ProfLogit fitness function — expected profit minus the L1 penalty — inside the GA-MaxAccLogit meta-learner is therefore the natural next step. Concretely, Equation (3.8)’s fitness becomes

$$\text{fitness}(\boldsymbol{\theta}) = \text{EMP}(\boldsymbol{\theta}) - \lambda \|\boldsymbol{\beta}\|_1,$$

where $\text{EMP}(\boldsymbol{\theta})$ is the expected maximum profit for credit scoring computed at the classifier’s decision threshold. Because the rest of the GA machinery (selection, crossover, mutation, elitism, soft-thresholding) is gradient-free, the only change required is in the fitness function. The Stacker would then optimise not the metric of *statistical correctness* but the metric the lender’s profit-and-loss statement cares about.

A second, more straightforward extension is to validate the present findings on a broader collection of credit datasets. The patterns observed here — the Stacker’s robustness, the

GHOST threshold sensitivity, the dominance of repayment-status features for default prediction — should hold on any sufficiently similar benchmark; testing on additional datasets would confirm the generality of the conclusion and would also increase the power of the Friedman and Wilcoxon-Holm post-hoc tests so that comparisons the current four-dataset benchmark could not resolve at $\alpha = 0.05$ become decidable.

With these two extensions the present benchmark would become a profit-aware, broadly validated credit-risk study — the natural next step beyond this thesis.

Appendix A

Python Code Listings

This appendix reproduces the principal Python listings used in the empirical work. All listings are runnable as written, given the libraries listed in Section 4.4.

A.1 Friedman Test on 16 Blocks

```
1 import numpy as np
2 import pandas as pd
3 from scipy.stats import friedmanchisquare, chi2, f as f_dist
4
5 MODELS = ['LogisticReg', 'KNN', 'SVM-RBF', 'DecisionTree',
6           'RandomForest', 'GradBoost', 'XGBoost', 'Stacker']
7 FS_ORDER = ['Full', 'RFE', 'Lasso-LR', 'Forward']
8
9 def build_matrix(score_dict):
10     rows, index = [], []
11     for ds in score_dict:
12         for fs in FS_ORDER:
13             index.append(f'{ds} | {fs}')
14             rows.append([score_dict[ds][fs][m] for m in MODELS])
15     return pd.DataFrame(rows, index=index, columns=MODELS)
16
17 def friedman_id(mat, alpha=0.05):
18     n, k = mat.shape
19     chi2_F, p_chi = friedmanchisquare(*[mat[m].values for m in MODELS])
20     dof_chi = k - 1
21     F_F = ((n - 1) * chi2_F) / (n * (k - 1) - chi2_F)
```

```

22     dof_F = (k - 1, (k - 1) * (n - 1))
23     p_F    = 1 - f_dist.cdf(F_F, dof_F[0], dof_F[1])
24     crit_chi = chi2.ppf(1 - alpha, dof_chi)
25     crit_F    = f_dist.ppf(1 - alpha, dof_F[0], dof_F[1])
26     return dict(n=n, k=k,
27                 chi2=chi2_F, p_chi=p_chi, dof_chi=dof_chi, crit_chi=crit_chi,
28                 F=F_F,      p_F=p_F,      dof_F=dof_F,      crit_F=crit_F)
29
30 # F1-macro and AUC matrices are constructed by build_matrix(...) above,
31 # from per-dataset score dictionaries. The Friedman + Iman-Davenport
32 # statistics for both metrics are then computed by friedman_id().

```

A.2 Wilcoxon Signed-Rank with Holm Correction

```

1 from itertools import combinations
2 from scipy.stats import wilcoxon
3
4 def wilcoxon_p(a, b):
5     if np.all(a - b == 0):
6         return 1.0
7     return float(wilcoxon(a, b, alternative='two-sided',
8                           zero_method='wilcox').pvalue)
9
10 def holm_chain(pairs, alpha):
11     pairs.sort(key=lambda x: x['p'])
12     nc = len(pairs)
13     key_sig = f'sig_{alpha:.2f}'
14     key_thr = f'thr_{alpha:.2f}'
15     rejected = True
16     for i, pr in enumerate(pairs):
17         thr = alpha / (nc - i)
18         pr[key_thr] = thr
19         if rejected and pr['p'] < thr:
20             pr[key_sig] = True
21         else:
22             rejected = False
23             pr[key_sig] = False
24     return pairs

```

```

25
26 def pairwise(mat):
27     pairs = []
28     for a, b in combinations(MODELS, 2):
29         p = wilcoxon_p(mat[a].values, mat[b].values)
30         ma, mb = mat[a].mean(), mat[b].mean()
31         if ma > mb:
32             winner, loser = a, b
33         elif mb > ma:
34             winner, loser = b, a
35         else:
36             winner, loser = 'tie', 'tie'
37         pairs.append({'a': a, 'b': b, 'mean_a': ma, 'mean_b': mb,
38                     'winner': winner, 'loser': loser, 'p': p})
39     holm_chain(pairs, 0.05)
40     holm_chain(pairs, 0.10)
41     return pairs

```

A.3 GA-MaxAccLogit Meta-learner

```

1 import numpy as np
2 from sklearn.metrics import f1_score
3 from sklearn.base import BaseEstimator, ClassifierMixin
4
5 class GAMaxAccLogit(BaseEstimator, ClassifierMixin):
6     """Logistic meta-learner whose coefficients are evolved by a
7     real-coded genetic algorithm to maximise F1-macro minus an
8     L1 penalty. Adapted from ProfLogit (Stripling et al., 2018)."""
9
10    def __init__(self, pop_size=60, n_generations=40,
11                 p_crossover=0.8, p_mutation=0.1,
12                 elite_frac=0.05, l1_lambda=0.01, seed=42):
13        self.pop_size = pop_size
14        self.n_generations = n_generations
15        self.p_crossover = p_crossover
16        self.p_mutation = p_mutation
17        self.elite_frac = elite_frac
18        self.l1_lambda = l1_lambda

```



```

19         self.seed          = seed
20
21     def _sigmoid(self, z):
22         return 1.0 / (1.0 + np.exp(-z))
23
24     def _soft_threshold(self, beta):
25         s = np.sign(beta) * np.maximum(np.abs(beta) - self.l1_lambda, 0)
26         return s
27
28     def _fitness(self, theta, X, y):
29         eta = X @ theta[1:] + theta[0]
30         prob = self._sigmoid(eta)
31         pred = (prob >= 0.5).astype(int)
32         f1m = f1_score(y, pred, average='macro')
33         pen = self.l1_lambda * np.sum(np.abs(theta[1:]))
34         return f1m - pen
35
36     def fit(self, X, y):
37         rng = np.random.default_rng(self.seed)
38         n, p = X.shape
39         # population of (p+1)-dim chromosomes; first gene = intercept
40         pop = rng.uniform(-3.0, 3.0, size=(self.pop_size, p + 1))
41         # soft-threshold non-intercept genes
42         pop[:, 1:] = self._soft_threshold(pop[:, 1:])
43         fit_pop = np.array([self._fitness(theta, X, y) for theta in pop])
44         n_elite = max(1, int(self.elite_frac * self.pop_size))
45
46         for g in range(self.n_generations):
47             # elite pool
48             elite_idx = np.argsort(-fit_pop)[:n_elite]
49             elite = pop[elite_idx].copy()
50
51             # fitness-scaled roulette selection
52             scaled = fit_pop - fit_pop.min() + 1e-6
53             probs = scaled / scaled.sum()
54             selected_idx = rng.choice(self.pop_size,
55                                     size=self.pop_size, p=probs)
56             selected = pop[selected_idx].copy()
57

```

```

58         # arithmetic crossover
59         for i in range(0, self.pop_size - 1, 2):
60             if rng.random() < self.p_crossover:
61                 w = rng.uniform(0.0, 1.0, size=p + 1)
62                 p1 = selected[i].copy()
63                 p2 = selected[i + 1].copy()
64                 selected[i] = w * p1 + (1 - w) * p2
65                 selected[i + 1] = (1 - w) * p1 + w * p2
66
67         # uniform random mutation
68         mut_mask = rng.random(selected.shape) < self.p_mutation
69         mutations = rng.uniform(-3.0, 3.0, size=selected.shape)
70         selected[mut_mask] = mutations[mut_mask]
71
72         # soft-threshold + re-evaluate
73         selected[:, 1:] = self._soft_threshold(selected[:, 1:])
74         fit_sel = np.array([self._fitness(theta, X, y)
75                             for theta in selected])
76
77         # elitism: replace weakest with elites
78         weakest = np.argsort(fit_sel)[:n_elite]
79         selected[weakest] = elite
80         fit_sel[weakest] = fit_pop[elite_idx]
81
82         pop, fit_pop = selected, fit_sel
83
84         # best chromosome
85         best = pop[np.argmax(fit_pop)]
86         self.intercept_ = best[0]
87         self.coef_ = best[1:]
88         return self
89
90     def predict_proba(self, X):
91         eta = X @ self.coef_ + self.intercept_
92         p1 = self._sigmoid(eta)
93         return np.vstack([1 - p1, p1]).T
94
95     def predict(self, X):
96         return (self.predict_proba(X)[:, 1] >= 0.5).astype(int)

```

A.4 Stacked Ensemble with Best-Meta Selection

```

1 from sklearn.model_selection import StratifiedKFold, cross_val_predict
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.ensemble import RandomForestClassifier,
   GradientBoostingClassifier
4 from sklearn.neighbors import KNeighborsClassifier
5 from sklearn.svm import SVC
6 from sklearn.neural_network import MLPClassifier
7 from xgboost import XGBClassifier
8
9 def build_meta_features(X_train, y_train, n_splits=5, random_state=42):
10     bases = [
11         ('rf', RandomForestClassifier(n_estimators=300, random_state=
12 random_state)),
13         ('gb', GradientBoostingClassifier(random_state=random_state)),
14         ('xgb', XGBClassifier(eval_metric='logloss', random_state=
15 random_state)),
16         ('knn', KNeighborsClassifier(n_neighbors=5)),
17         ('svm', SVC(kernel='rbf', probability=True, random_state=random_state
18 )),
19     ]
20     skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
21 random_state=random_state)
22     M = np.zeros((len(X_train), len(bases)))
23     for j, (name, model) in enumerate(bases):
24         M[:, j] = cross_val_predict(model, X_train, y_train, cv=skf,
25 method='predict_proba')[:, 1]
26     refit = [(name, model.fit(X_train, y_train)) for name, model in bases]
27     return M, refit
28
29 def best_meta_stacker(X_train, y_train, X_test, y_test):
30     M_train, refit_bases = build_meta_features(X_train, y_train)
31     M_test = np.column_stack([m.predict_proba(X_test)[:, 1]
32 for _, m in refit_bases])
33
34     candidates = {
35         'LR': LogisticRegression(max_iter=1000, random_state=42),
36         'RF': RandomForestClassifier(n_estimators=300, random_state=42),

```

```
34     'XGB': XGBClassifier(eval_metric='logloss', random_state=42),
35     'MLP': MLPClassifier(hidden_layer_sizes=(20,), max_iter=500,
36                           random_state=42),
37     'GA': GAMaxAccLogit(seed=42),
38 }
39 best_name, best_f1, best_meta = None, -1, None
40 for name, meta in candidates.items():
41     meta.fit(M_train, y_train)
42     pred = meta.predict(M_test)
43     f1m = f1_score(y_test, pred, average='macro')
44     if f1m > best_f1:
45         best_name, best_f1, best_meta = name, f1m, meta
46 return best_name, best_f1, best_meta
```

Bibliography

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [2] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, pages 785–794, 2016.
- [3] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.
- [4] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [5] Carmen Esposito, Gregory A. Landrum, Nadine Schneider, Nikolaus Stiefl, and Sereina Riniker. GHOST: Adjusting the decision threshold to handle imbalanced data in machine learning. *Journal of Chemical Information and Modeling*, 61(6):2623–2640, 2021.
- [6] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.
- [7] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [8] Eugen Stripling, Seppe vanden Broucke, Katrien Antonio, Bart Baesens, and Monique Snoeck. Profit maximizing logistic model for customer churn prediction using genetic algorithms. *Swarm and Evolutionary Computation*, 40:116–130, 2018.
- [9] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical Bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems*, 25, 2012.

-
- [10] Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937.
- [11] Dheeru Dua and Casey Graff. UCI machine learning repository. <https://archive.ics.uci.edu/ml>, 2019.
- [12] Ronald L. Iman and James M. Davenport. Approximations of the critical region of the Friedman statistic. *Communications in Statistics*, pages 571–595, 1980.